Approximate Bayesian Computation in Python

Erika Cule Juliane Liepe

April 8, 2009

Contents

1 Introduction **2**

2 Examples **4**

3 Using the package **18**

4 Installation **22**

5 Extending the library **23**

6 Troubleshooting **25**

7 Appendix **26**

Chapter 1

Introduction

The growing field of systems biology has driven demand for flexible tools to model and simulate biological systems. One established problem in the modelling of biological processes is the estimation of associated parameters. These parameters, such as kinetic constant and rates, are difficult to measure experimentally. A number of statistical approaches, both frequentist and Bayesian, have been proposed to estimate parameters given biological data and a proposed model. Where a number of putative models exist, a statistical method is required to discriminate between them.

Many approaches to the parameter inference problem return a single best estimate of the parameter value, where it would be preferable to obtain a probability distribution over the range of parameters. Recently, methods in Bayesian simulation (termed Approximate Bayesian Computation) have been employed, enabling the evaluation of a posterior distribution of parameters where the likelihood function is computationally intractable.

Here we present a Python package, `abc`, that implements parameter inference and model selection in dynamical systems. `abc` combines three algorithms (see Figure 1.1). The first is the ABC rejection sampler (Pritchard *et al.*, 1999). Starting from a given prior distribution, the ABC-Rejection method samples parameters and tests how well simulation of the model using these parameters fits to the experimental data. One disadvantage of this algorithm is that it is computationally expensive, particularly if the prior distribution is large. ABC-Rejection is automatically used when only one model and one maximal distance is given. The second algorithm, ABC-SMC for parameter inference, was described by Sisson *et al.* (2007). Instead of a single distance between the experimental and simulated data below which a sampled parameter set is accepted, ABC-SMC takes a series of maximal distances. Using a sequential Monte-Carlo approach, the posterior parameter distribution from one population is used as the prior for the next. By sampling parameters from the previous distribution, ABC-SMC progressively approaches the target distribution as the maximal distance decreases. Toni *et al.* (2009) showed that ABC-SMC could be extended to a model selection problem. Given several models, ABC-SMC for model selection and parameter estimation simultaneously evaluates posterior parameter distributions and allows for discrimination among sets of candidate models.

`abc` is designed to work with models written in Systems Biology Markup Language (SBML), a data exchange format based on XML. Models written in SBML can be obtained from the

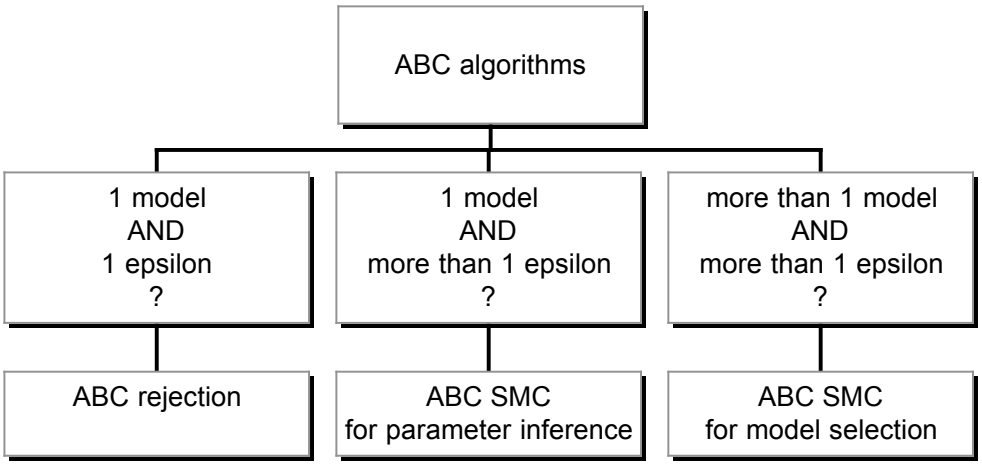


Figure 1.1: Schematic overview of the implemented ABC algorithms.

BioModels database (Le Novere *et al.*, 2006). Typical biochemical systems represented in the BioModels database include regulatory networks, metabolic networks and cell-signalling pathways. The twelfth release of the BioModels database contains 208 curated and 85 non-curated models. The `abc` package is able to analyze all dynamical models taken from the curated database that do not include algebraic rules. Non-dynamical models can not be investigated with this package. It is also possible to investigate other dynamical models that are not in SBML format, but this takes a more advanced usage of the package as a Python library.

The models can be treated as deterministic or stochastic models. To analyse deterministic models the package uses a modified version of the ODE solver implemented in Scipy, `odeint`. For stochastic systems a SDE solver and a Gillespie solver are implemented.

Chapter 2

Examples

We advise that you read through the Examples to get a feel for how the package works. The SBML models and user input files used in the examples are distributed with the package, enabling you to reproduce the examples yourself. Thus familiarised, you will be ready to use the package to perform ABC algorithms on your own models. For full instructions on how to use the package in a general case, see Chapter 3.

2.1 Model selection and parameter inference

We illustrate an example of the ABC-SMC algorithm for model selection using a range of simple models that describe the epidemiology of infections diseases. This example reproduces the findings of Toni *et al.* (2009). As described by Toni *et al.*, SIR models describe the spread of such disease in a population of susceptible (S), infected (I) and recovered (R) individuals (Anderson & May, 1991). The three models used for this example use systems of ordinary differential equations (ODEs) to describe the change in the number of S, I and R indivuduals over time.

Model 1

In the simplest case, the model assumes that every individual can be infected only once and that there is no time delay between the individual getting infected and their ability to infect other individuals.

$$\dot{S} = \alpha - \gamma SI - dS$$
$$\dot{I} = \gamma SI - \upsilon I - dI$$
$$\dot{R} = \upsilon I - dR$$

where $\dot{x}$ denotes the time derivative of x, $\mathrm{d}x/\mathrm{dt}$. Individuals, who are born at rate α, are susceptible (S). All individuals die at rate d, γ is the infection rate and υ is the recovery rate.

Model 2

To represent the delay between the time an individual gets infected and the time when they become infections, a population of individuals in a latent phase of infection, L, can be included in the model. Individuals in the latent population are infected but cannot infect others. Indiviuals make the transition from the latent phase to the infective phase at rate δ.

$$\dot{S} = \alpha - \gamma SI - dS$$
$$\dot{L} = \gamma SI - \delta L - dL$$
$$\dot{I} = \delta L - \upsilon I - dI$$
$$\dot{R} = \upsilon I - dR$$

Model 3

Another extension of the basic model allows the recovered individuals to become susceptible again at rate e.

$$\dot{S} = \alpha - \gamma SI - dS + eR$$
$$\dot{I} = \gamma SI - \upsilon I - dI$$
$$\dot{R} = \upsilon I - dR - eR$$

First, get your models...

The package `abc` is primarily written to handle SBML models. The BioModels Database (Le Novere *et al.*, 2006), a database of SBML models, contains mathematical models of biological interest. These SIR models, however, are not available from the BioModels database, so SBML models have been constructed for the purpose of this example. To construct the models, we make use of SBML shorthand and its conversion tools (Gillespie *et al.*, 2006), and construct the three SIR models, `SIRmodel1.xml`, `SIRmodel2.xml` and `SIRmodel3.xml`.

...then prepare for model selection

Two scripts are included in the package `abc` to automate the most common applications of the modules in the package. To use them, these scripts, `getSBMLsummary.py` and `abcScript.py` must be copied into the same folder as the SBML model(s) that will be used when using the package. As can be seen from Figure 2.1, `getSBMLsummary.py` is the first script to use. At the command line, type

```
$ python getSBMLsummary.py SIRmodel1.xml SIRmodel2.xml SIRmodel3.xml
```

In your working directory, two new files will be written: a summary of the model(s), `model_summary.txt`, and a template file, `input_file_template.txt`.

Inspection of `model_summary.txt` will reveal a summary of the properties of the model(s), including the numbers of species and parameters contained therein, how these are handled by the simulators, and the presence of any rules, events and functions in the model.

`model_summary.txt` is primarily for information and for verification that the properties of your model are as expected.

`input_file_template.txt` contains, or will contain, all the information to be passed to the ABC-SMC algorithm. It might be prudent to rename this template file with a more meaningful name.

```
$ mv input_file_template.txt input_file_SIR.txt
```

`input_file_template.txt` is written with its own comments in it, preceeded by `#` (as in Python). These comments should help the user to complete the template; additional comments can be added, preceded by `#`.

`abcScript.py` has the capacity to simulate the model (see Example 2), to generate a synthetic data set, or to perform the ABC-SMC algorithm using the information in the input file.

Editing the input file

Some of the information in the input file does not need to be changed.

modelnumber Integer, the number of models described in the template.

restart Boolean, to describe whether or not we wish to restart the simulation. This is covered in more detail in Example 2. For now, we leave it as `False`.

beta The number of times to simulate an SDE model before determing if a sampled parameter set should be accepted. As all of our models are ODE models, this will be ignored.

To perform ABC-SMC, some additional information does need to be added to the input file.

epsilons A comma-delimeited list of floats, these are the distances below which a parameter set is accepted by the algorithm.

population size Integer, the size of each population.

dt Float, the internal timestep. For a stiff model, a small **dt** is required for a successful simulation. However, the smaller the value of **dt**, the longer the simulation will take. The SIR models are not particularly stiff so **dt** can be set to 1.

rtol, atol Floats, relative and absolute error tolerances for the ODE solver. These are optional and by default are commented out in the input file. The SIR models are not particularly stiff so we leave these commented out. If a model cannot be simulated, setting these small may be required for a successful simulation.

data The data has two aspects: the concentrations or molecule numbers of the interacting species, and the corresponding timepoints. Both the timepoints and the data themselves should be entered as a comma-delimited list of floats, starting with the initial values and time 0.

Some model-specific information must be completed for each of the models.

name The name of the python file that will represent the model. Defaults to `model1`, `model2`, ... ,`modelN`. These have been renamed in the input file we are using.

source The name of the .xml file containing the SBML model. The source file(s) must be in the same working directory as the scripts and the input file.

type The integration type used to simulate the model. Defaults to `ODE`. Other integration types are `SDE` (see Example 2) or `Gillespie` (see Example 3).

Initial values A comma-delimeited list of floats, these are the initial values for the species whose trajectories are described by the model. By default, these values are taken from the SBML file.

fit A string, defaults to `None`. If **fit** is not `None`, it must take the form of a comma-delimited list of fitting instructions describing how to fit the species in the SBML model (denoted `species1, species2, ... , speciesN`) to the variables given in the **data**. Model 2 in this example describes the trajectories of four species, including the latent population L. However, our data only describes three variables. We give the fitting instruction

```
fit: species1, species3, species4
```

to denote that the first species in Model 2 describes variable 1 in the data, the third species in Model 2 describes variable 2 and the fourth species in Model 2 describes variable 3. Fitting instructions can include simple arithmetic operations. For example if data represent the sum of the first two species in a model the fitting instruction for that variable should be written `species1+species2`.

Priors For each parameter we specify the distribution:

- Constant, value
- Uniform, lower, upper
- Lognormal, location, scale.

kernels For each parameter we also specify the pertubation kernel:

- uniform, a, b: uniform distribution on the interval $[a, b]$
- gauss, a, b: gaussian distribution with mean a and variance b

If we do not use the additional keyword "constant kernels" we only need to provide the type of the pertubation kernel, because the location and scale will be computed after each population.

Run model selection

To recap, in the current working directory we have

- Three SBML models to select from, `SIRModel1.xml`, `SIRModel2.xml` and `SIRModel3.xml`
- The completed input file, `input_file_SIR.txt`

- The scripts `getSBMLsummary.py`, which we have already used, and `abcScript.py` which we will use now.

`abcScript.py` has a number of options. To see a list of these, use the inbuilt help function.

```
$ python abcScript.py -h
```

By default, `abcScript.py` requires one argument, `--infile filename.txt`. So to use our completed input file to run ABC-SMC, at the prompt we type

```
$ python abcScript.py --infile input_file_SIR.txt
```

Depending on the stiffness of the model(s), the length of each simulation, **dt**, the priors, epsilons and population size, ABC-SMC may take from minutes to hours to run. To monitor the progress of the algorithm, run `abcScript.py` with full output:

```
$ python abcScript.py --infile input_file_SIR.txt --fulloutput
```

Outputs from ABC-SMC

The outputs from running the ABC-SMC algorithm are saved in your working directory.

- `_data.pdf`, a scatter plot of your input data. (Figure 2.2).
- A folder, `_results_`, containing the following files:
  - `ModelDistribution_1.pdf`. Histograms of the posterior distribution of accepted models after each population (Figure 2.3). Above each histogram the population number, epsilon, and acceptance rate for that population are displayed.
  - Two text files, `ModelDistribution.txt` and `rates.txt`, containing the same information.
  - One text file per population, `distance_PopulationN.txt`, listing the distances of the accepted particles together with the model number of the accepted model.
- `_results_` also contains one sub-folder per model. These sub-folders, suffixed with the model name (in this case `_resultsSIRmodelN`), contain sub-folders for each population, `population_N`. Each contains:
  - `data_PopulationN.txt`, the accepted parameter sets
  - `ScatterPlotPopulationN.pdf`, scatter plots of all accepted parameters. (See Figure 2.4)
  - `TimeserioesPopulationN.pdf`, simulations of the model using ten accepted parameter sets, to compare with the data. Refer to Figure 2.5.
  - `weightedHistograms_PopulationN.pdf`, histograms showing accepted parameter distributions.
  - `data_weightsN.txt`, the weights of the accepted parameters.

- `copy` contains in binary data form the information required to restart the ABC-SMC algorithm using the last population. These files are not human-readable but are read into Python if the algorithm is being run restarting from a previous population. See Example 2.

In this example, the data (which are available in `input_file_SIR.txt`) are generated from Model 1 and perturbed with Gaussian noise, $N(0, (0.2)^2)$. From the posterior distributions (Figure 2.3), after eleven populations, the correct model has been selected 1000 times in a population of 1000 particles.

2.2 Parameter inference - ABC-SMC

As shown in Figure 2.1, if only one model is used, in the absence of model selection the package `abc` effectively performs ABC-SMC for parameter inference.

In this section we perform ABC-SMC for parameter inference using the linear growth model.

This model is an instance when the flexibility of the package is useful. The noise terms in the Python representation of the model automatically generated by the program are too large for the model to be simulated efficiently. (See Chapter 6). In this example the Python file has been edited to reduce the magnitude of the noise terms. The script `abcScript.py` has been modified accordingly, such that the edited Python file representing the model is not overwritten when the script is run.

`abcScript.py` has the capacity to generate a synthetic data set from the model. To do this, a series of timepoints at which data are required must be added to the input file, and all the parameters in the input file must be constant. In this example, the data were simulated from the model parameterised with parameters $10.0, 1.0$.

```
$ abcScript_modified.py --infile input_file_simulation -y
```

In the input file, we input the data and the initial epsilon schedue, $\epsilon = \{5, 4, 3\}$. The population size is 500.

To run our script, we type

```
$ python abcScript_modified.py --infile input_file_lingrow.txt
```

After running the script, we inspect the outputs. From Figure 2.6) we see that the accepted parameters span the entire prior distribution. Our choice of ϵ was too large for meaningful parameter inference. Using the **Restart** option, we can adjust the epsilon schedule and restart the program, making use of the posterior distribution from the last population. To do this, we make a copy of the user input file.

```
$ cp input_file_lingrow.txt input_file_lingrow2.txt
```

In the new user input file, we set the **Restart** flag in the input file to `True` and change the epsilon schedule. We can then restart the program. The data in the binary data files saved in the folder `copy` will be read into python and used as the posterior distribution from which the parameters are sampled.

```
$ python abcScript_modified.py --infile input_file_lingrow2.txt
```

By inspecting the output, we can see that after restarting the program with a more appropriate epsilon schedule, more meaningful parameter inference is obtained.

2.3 Parameter inference - ABC-Rejection

In an even simpler case than that described in Example 2, specifying a single epsilon will return a single population of accepted parameters, effectively implementing the ABC Rejection algorithm (Pritchard *et al.*, 1999). We give an example using the dimerisation model of Wilkinson (2006). This model has been written with appropriate rate laws for a discreet stochastic simulation using Gillespie's direct method (Gillespie, 1977).

As in Example 2, in this example the data are data from the model (Figure 2.8). The procedure for using `abc` to implement the ABC-Rejection algorithm follows that of the use of the package to implement ABC-SMC. By specifying a single epsilon together with a size for the population of accepted particles, a single population of accepted parameters is returned.

Figure 2.9 shows the posterior distributions of the two parameters in the dimerisation model. The true parameter values were parameter 1, $1e^{-15}$, parameter 2, $1.66e^{-3}$. As discussed by Toni *et al.* (2009) and Sisson *et al.* (2007), the ABC-Rejection algorithm is less efficient than ABC-SMC. With the ability to restart the algorithm using the posterior distribution of a population of accepted parameters, the user can transition from ABC-Rejection to ABC-SMC to narrow the posterior distribution further with further iteration(s) of ABC.

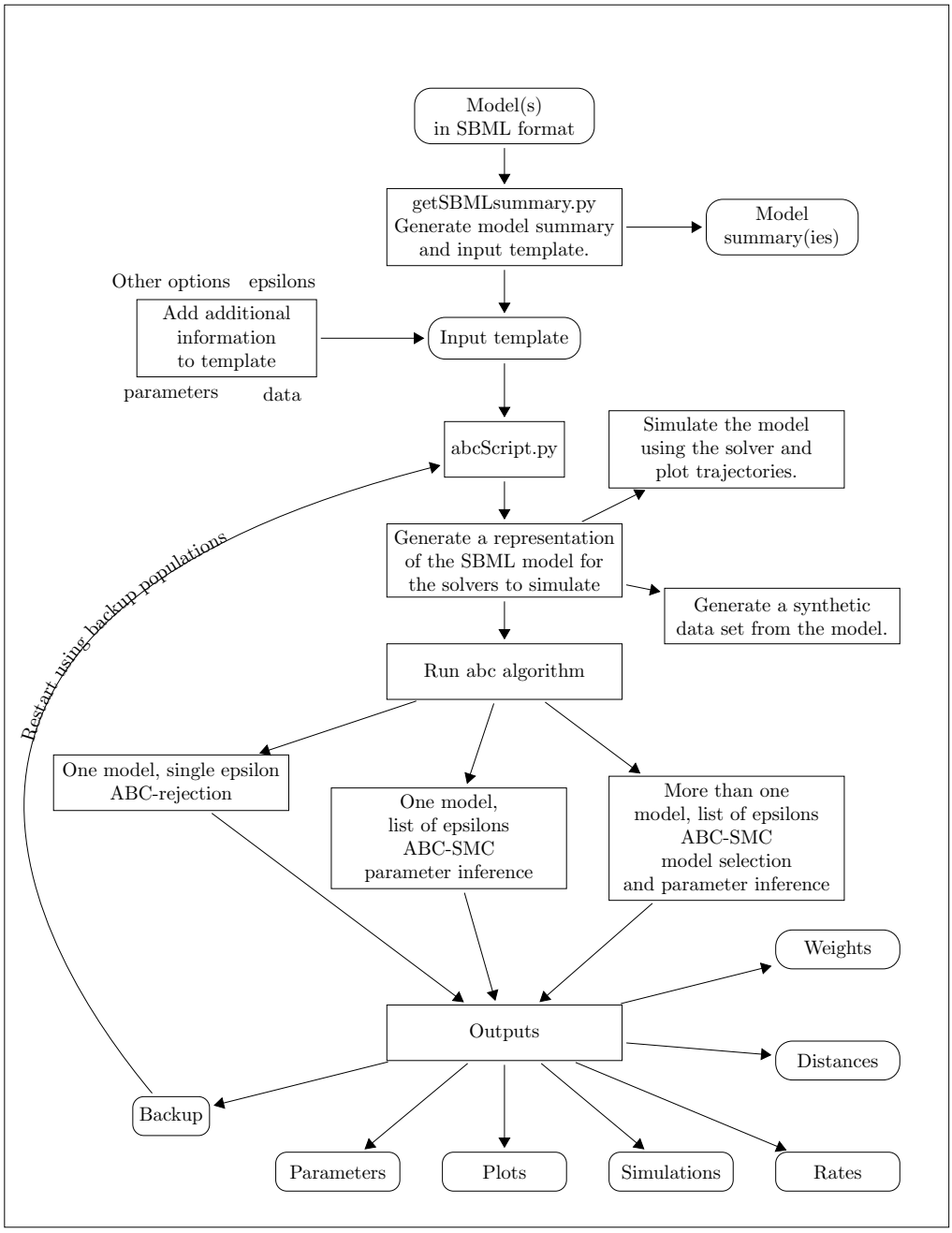


Figure 2.1: A user's-eye-view of the package abc.

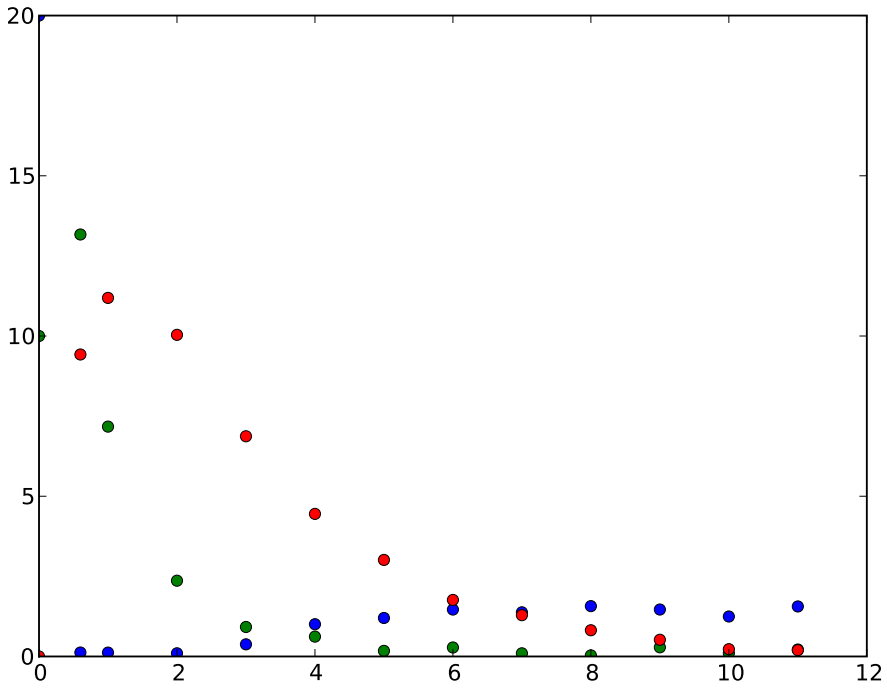


Figure 2.2: Scatterplot of data from an SIR model as input to ABC-SMC.

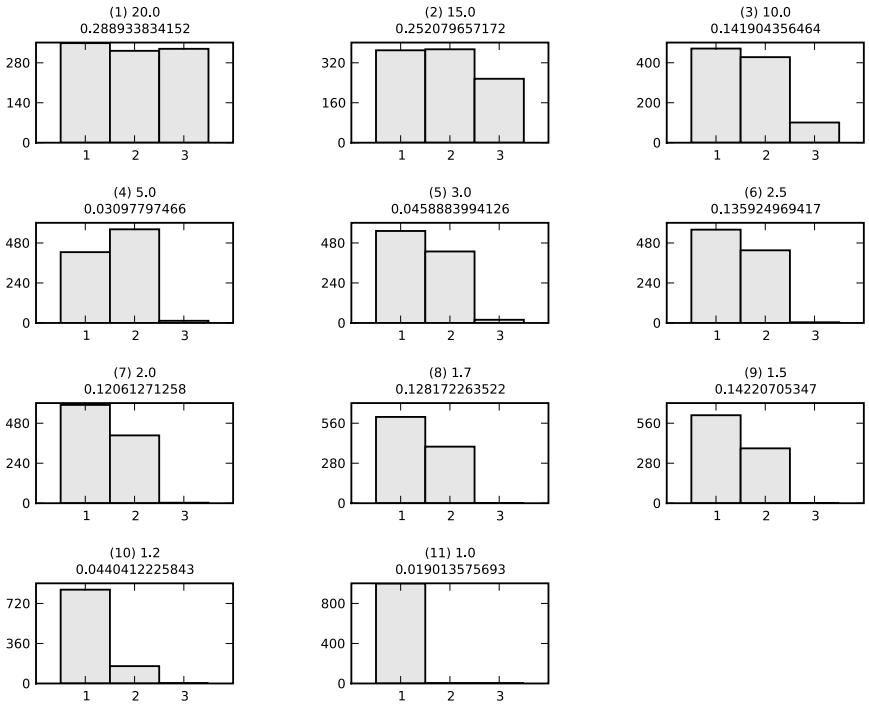


Figure 2.3: Posterior distributions of models after ABC-SMC using the three SIR models.

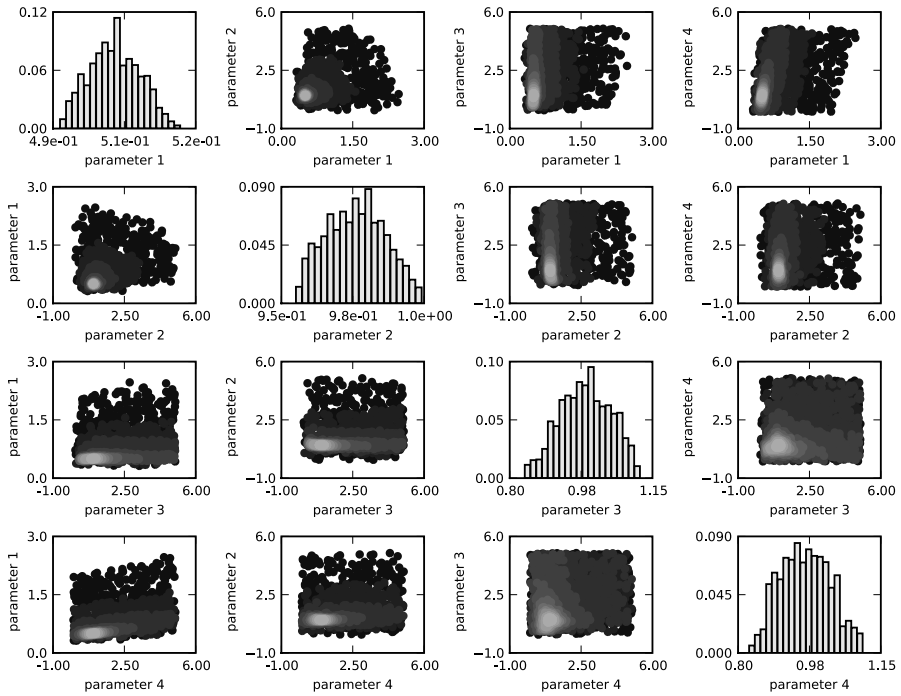


Figure 2.4: Posterior distributions of parameters of SIR Model 1, final population.

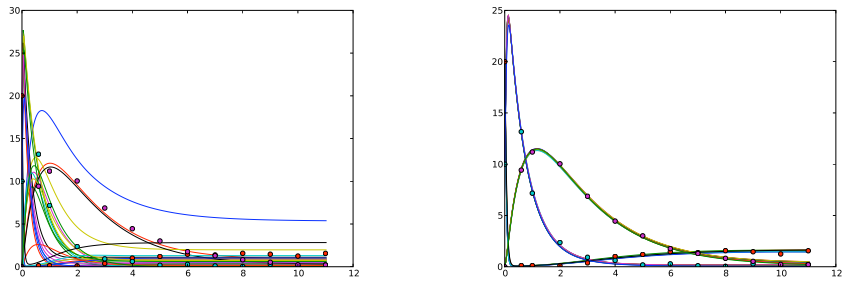


Figure 2.5: Simulations of SIR Model 1 using accepted parameters from the first (left) and last (right) populations.

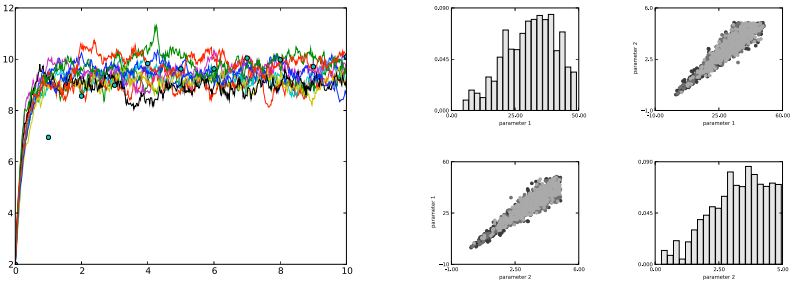


Figure 2.6: ABC-SMC for parameter inference using the linear growth model. Results after the third population ($\epsilon = 3.0$). Left, simulations using ten accepted parameter sets. Data are turquoise circles. Right, posterior parameter distributions.

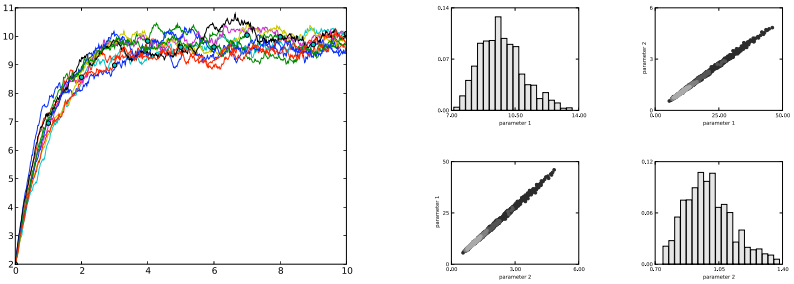


Figure 2.7: Restarting ABC-SMC for parameter inference using a modified epsilon schedule allows us to more closely estimate the parameter distribution. Here we show results of the final population after restarting the algorithm ($\epsilon = 1.0$). Left, simulations of the linear growth model using ten accepted parameter sets. Data are turquoise circles. Right, posterior parameter distributions.

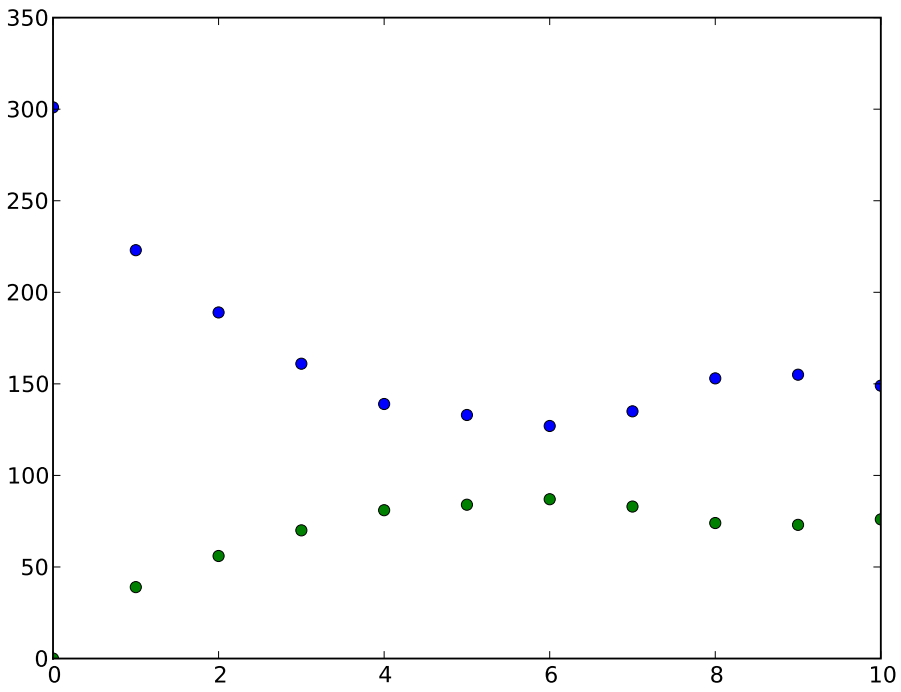


Figure 2.8: Simulation of the dimerisation model of Gillespie (1977) using "Gillespie's Direct Method". These data are used in the ABC-Rejection algorithm.

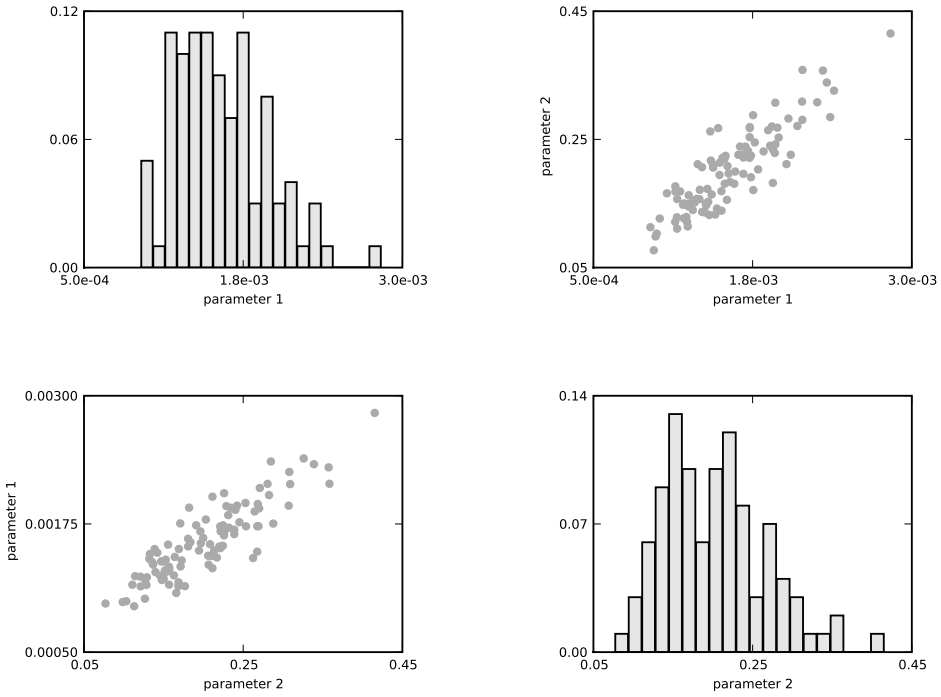


Figure 2.9: Posterior parameter distributions of parameter 1 and parameter 2 in the dimerisation model.

Chapter 3

Using the package

`abc` can be used as a standalone Python library. However, facilitate the most common applications of the package, `abc` is supplied with two scripts, `getSBMLsummary.py` and `abcScript.py`. These scripts combine the functions in the package to implement the ABC algorithms.

`abcScript.py` implements parameter inference and model selection using the ABC-SMC algorithm (Toni *et al.*, 2009), as well as the ABC rejection sampler (Pritchard *et al.*, 1999). The program works directly with models written in SBML format.

3.1 Model conversion from SBML to Python format

When `abcScript.py` is run, model(s) written in SBML format are used to generate an appropriate Python module representing the model, via a call to `importSBML`. The format of the Python module written depends on the integration type, which also informs the program which solver to use to simulate the model. (See Section 5.2.)

In an SBML model, values can be assigned to compartment sizes, to parameters and to species. When the Python module representing the model is written, the model is inspected to determine how values should be assigned during the course of the simulation. Compartments typically have a constant size and are considered as an additional parameter to the model, albeit one that the user may not want to infer. Some species may be outside of the scope of the reactions in the model, having either a constant value or a value assigned by an assignment rule. A species that has its value assigned in this way is also treated as a parameter to the model.

Conversely, SBML has the capacity for rate rules to be used to assign values to species, parameters or compartment sizes. Rate rules are time-dependent rules used to assign values to variables. Because our solvers typically assign values to species over time to compute a trajectory, species, parameters and compartment sizes with rate rules are treated as 'species' in a Python module representing the SBML model.

3.2 The user input file

Not all the information required for the algorithms is intrinsic to the model, therefore a text file that contains the additional information (such as initial conditions and parameter values) must be supplied to the script. This text file, the "user input file", must be written in a specific format. Examples are included in the "Examples" folder which is supplied along with the package; details of the input file are given here.

Some information is general to the algorithm.

modelnumber Number of the model for which details are described in this input file

epsilon Comma-delimited list of maximum distances between the data points and the simulated trajectories. If the length of the epsilon series is one and only one model is analyzed, the algorithm implemented is the ABC-Rejection sampler.

population size Number of particles to be accepted.

data This section is devided into 'times' and 'variable'. Both describe the experimental data for which the parameters or models respectively have to be analyzed. The time points are given in 'times' as a comma-delimited list starting with 0. The species concentrations (in the case of an ODE or SDE simulation) or molecule numbers (in the case of a Gillespie simulation) are also given as a comma-delimited list and denoted as variable1, variable2, ..., variableN for N different species.

dt The internal time step for the solvers.

beta Number of times to simulate each sampled parameter set. For deterministic systems beta is set to 1. For analysis of stochastic systems beta should be chosen larger than 1.

Some optional keywords can be supplied.

rtol, atol For models to be simulated as an ODE system these two keywords can be used to set the relative and absolute error tolerances for the numeric simulation. For stiff models, this may be necessary for successful simulation.

restart Frequently in the implementation of the ABC-SMC algorithm, the epsilon schedule selected in the first instance might be sub-optimal, leading to a high acceptance rate and too wide a posterior distribution. In addition this makes parameter inference computationally expensive. To avoid wasting the information from initial attempts at parameter inference, it is possible to make a backup that stores the information about each popualation after it has been completed. With this backup one can stop the program, change the maximum distances or any other parameters and restart the program with the results of the last population. To do this set: `<restart>: True` When restarting from a backup population, it is important not to increase the population size and to keep the structure of the models constant. Permitted changes include **epsilon**, **beta**, **dt**, **rtol** and **atol**, the values in **data** (but not the structure), the initial concentrations, the prior distributions (for constant parameters) and the pertubation kernels. Which of these changes will make the inference more informative, we will leave the user to decide.

For each model in the input file, additional information must be supplied.

model i ($i = 1, \ldots, M$, where M is the total number of models to be investigated.) For ABC-Rejection and ABC SMC for parameter inference, $M = 1$.

name The name of the Python file that will be written to represent the model in a format that is interpretable by the solvers.

source The name of the .xml file containing an SBML representation of the model.

type The simulation type. One of `ODE`, `SDE` or `Gillespie`.

Initial values A comma-delimited list of initial values for the model. This list should have the same dimensionality as that of the model.

parameters Parameters are described with two properties:

- **prior** describes the prior distribution. Implemented prior distributions are
  - constant, x: constant parameter with value x
  - uniform, a, b: uniform distribution on the interval $[a, b]$
  - lognormal, a, b: lognormal distribution with location a and scale b
- **kernel** The implemented ABC SMC algorithms compute the pertubation kernels after each population, dependent on the marginal parameter distributions. Therefore it is necessary to provide the type of distribution. However, it is also possible to use constant pertubation kernels. In the last case the pertubation kernels need to be defined with three parameters: type, location and scale. Implemented distributions for the pertubation kernels are
  - uniform, a, b: uniform distribution on the interval $[a, b]$
  - gauss, a, b: gaussian distribution with mean a and variance b

One model-specific keyword is optional.

fit Denotes the correspondence between the species defined in the SBML model and the experimental data. If this keyword is not given, or if **fit** is `None`, all the species in the model are fitted to the data in the order that they are listed in the model. Otherwise, a comma-delimited list of fitting instructions with the same length as the dimensions of the data can be supplied. Simple arithmetic operations can be performed on the species from the SBML model. To denote the `N`th species in the SBML model, use `speciesN`. For example, to fit the sum of the first two species in the model, write **fit**: `species1+species2`.

This user input file can be written by hand; however, we forsee attempts to hand-write the user input file leading to errors. Therefore, the script `getSBMLsummary.py` is provided. This script generates two text files. The first one is a summary of all SBML models to be investigated. The second one is a template for the user input file. This template includes all necessary keywords with instructions how to fill in the information. It has already the correct structure for the models and provides default values for several keywords.

Beside implementing the ABC SMC algorithms the program `abcScript.py` provides an easy way to simulate a biochemical system directly from the SBML source without converting it manually into another Python compatible format. Simulations works with requires the user input file, but less information is required. For a simulation, **modelnumber**, **dt** and **times** must be supplied. For each model, **source**, **type**, **initial values** and **parameter** must be provided; all parameters must have a constant prior distribution for a simulation.

Both `abcScript.py` and `getSBMLsummary.py` can be run from the command line. To run `getSBMLsummary.py` with three SBML models, type

```
$ python getSBMLsummary.py source1.xml source2.xml source3.xml
```

To run `abcScript.py`, use

```
$ python abcScript.py -i user_input_file.txt
```

When running `abcScript.py`, several options are possible. To see a list of these, type

```
$ python abcScript.py --help
```

For example to run abcScript with the user input file 'MyInputFile', to have a full output of results on the shell and to avoid diagnostic plots after each population type

```
$ python abcScript.py -i 'MyInputFile' --fulloutput -d
```

```
-d  --diagnostic  no printing of diagnostic plots
-s  --save        no backup after each population
-t  --timeseries  no plotting of simulation results after each population
-p  --plotdata    no plotting of given data points"
-i  --infile      declaration of the input file.  This input file has to be provided
                  to run the program!
-y  --synthetic   synthetic data set at the timepoints given in the input file, using
                  initial conditions and constant parameters.
-S  --simulate    Simulate the model over the range of timepoints,
                  using initial conditions and constant parameters.
-f  --fulloutput  print epsilon, sampling steps and acceptence rates after each population.
-h  --help        Print this list of options.
```

Table 3.1: Full list of options for the script `abcScript.py`

Chapter 4

Installation

The package `abc` provides a Python library containing function for parameter inference and model selection (Toni *et al.*, 2009; Pritchard *et al.*, 1999) of dynamical systems taken from the Biomodels database. Together with the scripts `getSBMLsummary.py` and `mainScript.py` it creates a user-friendly tool. It is advisable to use the package together with the Python Enthought Distribution. It works on both operationg systems MacOS and Linux. The package has to be installed in the Python path. We recommend that you install the package in the same directory as other Python libraries. For example, using the recommended Enthought Python Distribution, the package should be installed in:

```
.../lib/python2.5/site-packages/abc/
```

Now the `abc` package can be used like any other Python library. To use the two programs `getSBMLsummary.py` and `abcScript.py` copy and paste them into the working directory. They are used as ordinary Python scripts.

`abc` has the below dependencies. With the exception of libSBML, all of these are included in the Enthought Python Distribution

```
libSBML
math
matplotlib
Numpy
os
pickle
re
sys
Scientific
Scipy
```

Chapter 5

Extending the library

5.1 Using the modules independently of the scripts

`abc` can be imported into an interactive Python session. By defining the arguments to the functions in the package in the interactive namespace, the functions can be used in the Python shell.

5.2 Writing Python modules to represent models

Examples of the format of Python modules that are compliant with ODE, SDE and Gillespie solvers are supplied with the package. These are the models that were genarated by the package when the Examples were prepared (see Section 2).

Broadly, we use the function definition `modelfunction` to define the callable function solved by the ODE and SDE systems. Functions passed to the ODE solver return the derivatives of the species whose trajectories are described by the function; models for simulation by the SDE solver also return the noise terms for each species. An additional function, `rules`, is obligatory even if empty. `rules` takes as arguments species, parameters and the time, `t`, and returns the species and parameters. The `rules` function and can contain time-dependent rules or events. User-defined functions required by the model can be defined at the top of the module and called within it.

Models for the Gillespie solver are written with one function, `Hazards`, which returns the hazards for each reaction.The reactions themselves are defined as `reaction0`, `reaction1`, ..., `reactionN`. Each reaction takes all species as arguments and returns all species, unchanged if these are not changed by the reaction. A dictionary, defined `Switch`, is used by the solver to call the chosen reaction.

5.3 Defining your own distance functions

The module `euclidian` contains a function, `euclidianDistace`, which returns the Euclidian distance between two arrays of equal dimensions. The advanced user could define his own

distance function, either in the `euclidian` module or elsewhere. The distance function is called in the `abcSimulator` function within the `abcSMC_model` module; the user would need to call his own distance function to compare the experimental and simulated data.

5.4 Defining your own priors

Parameters are sampled from priors or from previous distributions in the module `abcSMC_model`, via the functions `sampleParametersFromPrior` and `sampleParametersFromPrevious`, which are called in the function `sampleTheParameter`. Prior distribution definitions are also required as an input to `computeParticleWeights`. If the user wishes to define their own prior distributions, a sensible way to do this would be to re-define one of the existing prior distributions with the required prior. The user should be aware that user-defined priors will not be supported by `abcScript.py`.

5.5 Defining your own kernels

Perturbation kernels are used to perturb sampled parameters in the function `sampleParametersFromPrevious`, which is called in the function `sampleTheParameter`. If the user wishes to define a new perturbation kernel, a sensible way to do this would be to overwrite one of the existing kernel definitions with the required kernel. The user should be aware that user-defined kernels will not be supported by `abcScript.py`.

Chapter 6

Troubleshooting

Before the ABC algorithm is implemented, the package checks that all the supplied information is consistent with each other and with the model(s) being used. If a discrepancy is found an error message is returned and the program is stopped. Despite this, some known issues remain.

- When attempting to simulate a stiff ODE model, the error message

  `Excess work done on this call. Perhaps wrong Dfun type.`

  may be encountered. This error message arises when the model is too stiff to be solved successfully by the `odeint` solver from Scipy. If this error message is encountered, specifying a smaller **dt** and/or a small **rtol** or **atol** (on the order of $1e^{-5}$ to $1e^{-7}$) may allow a stiff model to be successfully simulated; however, such a stiff model may take such a long time to be simulated as to be impractical for ABC-SMC.

- When attempting to simulate a model using the SDE solver, the message

  `SDE simulation failed. Try a smaller dt.`

  indicates that the concentrations of species in the simulation have fallen below zero. At this point the simulation is stopped and, if this simulation is occuring in the context of ABC-SMC or ABC-Rejection, the sampled parameters are rejected. If this error is encountered, a smaller **dt** may enable a successful simulation by having the effect of making the noise at each step smaller. Another means of simulating the model with less noise is to edit the Python function representing the model; in this case the user will have to edit the script `abcScript.py` to ensure that the edited model is not overwritten by a call to `importSBML`.

Chapter 7

Appendix

7.1 Package information

Package Name

abc

Package Description

The abc package @ 2009
Erika Cule and Juliane Liepe
Centre for Bioinformatics
Imperial College London

Package Contents

GillespieAlgorithm
SBMLparse
abcSMC_model
abcodesolve
checkInputArguments
euclidian
generateTemplate
getResults
parseInfo
relations
sdeint

7.2 Module and function details

Module names are given in **bold type**; function names are given in *italics*.

7.2.1 GillespieAlgorithm

select_hazard(total_hazard,list_of_hazards)

```
chose a hazard from the list by by hazard weights.

***** args *****

total hazard:

    Sum of hazards

list of hazards:

    List of hazards, one for each reaction
```

GillespieInt(func, initValues, parameters, outputtimes)

```
Simulate a function via Gillespie's direct method.
Return numbers of molecules at the times in outputtimes.

***** args *****

func:

        function to integrate. generated by parseInfo.py from an SBML model
        (or equivalently formatted)

initValues:

        a list of floats representing the initial values of the species
        whose trajectories are described by func.

parameters:

        a tuple of parameters to pass to func

outputtimes:

        a list of times at which to return data points.
```

7.2.2 SBMLparse

write_GillespieFunctions(stoichiometricMatrix, kineticLaw, numSpecies, numReactions, species, parameterId, InitValues, speciesId,name, listOfFunctions, FunctionArgument, FunctionBody, listOfRules, ruleFormula, ruleVariable, listOfEvents, EventCondition, EventVariable, EventFormula)

```
Write the Python function representing the model for simulation using
GillespieInt (simulates the model using Gillespie's direct method)
```

write_ODEFunctions(stoichiometricMatrix, kineticLaw, species, numReactions,speciesId,listOfParameter, parameterId,parameter,InitValues,name, listOfFunctions, FunctionArgument, FunctionBody, listOfRules, ruleFormula, ruleVariable, listOfEvents, EventCondition, EventVariable, EventFormula)

```
Write the Python function representing the model for simulation using
odeint (the ODE solver)
```

write_SDEFunctions(stoichiometricMatrix, kineticLaw, species, numReactions,speciesId, listOfParameter,parameterId,parameter,InitValues,method,name, listOfFunctions, FunctionArgument, FunctionBody,listOfRules, ruleFormula, ruleVariable, listOfEvents, EventCondition, EventVariable, EventFormula)

```
Write the Python module representing the model for simulation using
sdeint (the SDE solver)
```

importSBML(source,integrationType,ModelName=None,method=None)

```
Import one or more SBML models.
Obtain all the information from the model(s) required to write the Python
module representing the model.
Make a call to the appropriate writing function.

***** args *****

source:

        List of strings.
        Each entry is the name of an SBML file to be parsed.

integrationType:

        List of strings, the same length as source.
        Each entry defines the algorithm the model will be simulated
        One of:
        ODE             ---     for deterministic systems; solved with odeint (scipy)
        SDE             ---     for stochastic systems; solved with sdeint (abc)
        Gillespie       ---     for staochastic systems; solved with GillespieAlgorithm (abc)

***** kwargs *****

ModelName:

        List of strings.
```

```
        Names of Python module(s) representing the model(s).

method:

        Integer.
        Type of noise in a stochastic system.
        Only implemented for stochastic systems solved with sdeint.
        Options:
        1 --- default
        2 --- Ornstein-Uhlenbeck
        3 --- geometric Brownian motion
```

7.2.3 abcSMC_model

abcSimulator(ModelName, data, timepoints, numOutput, epsilon, InitValues, integrationType, modelWeights, priors, kernel, fit=None, sampleFromPrior=False, beta=1, dt=0.01, restart=False, source=None, diagnostic=True, pickling=True, plotDataSeries=True, full=False, constKernels=False, rtol=None, atol=None)

```
The simulator for the abc algorithm:

***** args *****

ModelName:

          List of strings, where each string represents a name for a model.
          The length of this list is determind by the number of models that
          are investigated.

          Example:
          To investigate 3 models with the names "model1", "model2" and "model3":
          ModelName = ('model1','model2','model3')

          To investigate one model with the name "MyInterestingModel":
          ModelName = ('MyInterestingModel',)

data:

          List of floats that coresponds to the experimental data.
          The length of this list is determind by the length of the argument timepoints.

timepoints:

          List of floats.
          The length must not differ to the length of data. Each timepoint describes an
```

external timepoint, where data are available.

numOutput:

Integer.
numOutput corresponds to the population size, i.e. how many particles must be accepted in each population.

epsilon:

List of floats.
Epsilon corresponds to the maximum distance between experimental and simulated data. The length of this list corresponds to the number of populations.

Example:
maximum distance for ABC Rejection (only one population):
epsilon = (1.5,)
maximum distance for ABC SMC with five populations:
epsilon = (3.5,3.0,2.5,2.0,1.5)

InitValues:

2D list of floats.
The first dimension describes each model, the second dimension describes the initial condition for each species in the particular model.

Example:
2 models, the first model has 3 species, the second model has 4 species:
InitValues = [[1.9 , 20 , 32.1] , [2.0 , 1.3 , 2.4 , 3.4]]
1 model with only one species:
InitValues = [[2.0] ,]

IntegrationType:

List of strings.
The length of this list is determind by the number of models to be investigated.

Possible strings are:
ODE --- use the odeint (Scipy) to simulate a deterministic system
SDE --- use the sdeint (abc) to simulate the system with SDEs
Gillespie --- use the GillespieAlgorithm (abc) to simulate a stochastic system

Example:
1 model to be simulated with SDEs
IntegrationType = ('SDE' ,)

3 models, first two deterministic, third stochastic via Gillespie
IntegrationType = ('ODE' , 'ODE' , 'Gillespie')

modelWeights:

List of floats.
This list describes the discrete prior distribution of the models.The length of the list is determined by the number of models to be investigated. The entries of the list display the weights for each model, starting from a uniform distribution. These weights do not need to be normalized to 1.

Example:
1 model
modelWeights = (1,)
5 models with equal weights
modelWeights = (1,1,1,1,1)
5 different weighted models
modelWeights = (2,1,4,3,7)

priors:

3D list.
The first dimension represents the number of models, the second dimension represents the number of parameter for a particular model and the third dimension represents the distribution for this parameter and has a constant length of 3. The first entry for each parameter is an integer number that stands for a specific distribution.

Implemented distributions are:
0 --- constant parameter.
Example: constant parameter with value 12.3
[0 , 12.3 , x] , where x can be any number

1 --- uniform distribution.
Example: uniform distribution in the range 0.1 to 50
[1 , 0.1 , 50]

2 --- lognormal distribution.
Example: lognormal distribution with mean 3 and varianz 1.5
[2 , 3 , 1.5]

Example:
1 model with 3 parameter, the first two parameter have uniform prior between 0 and 5, the third parameter has lognormal prior with mean 1 and variance 3.
[[[1,0,5],[1,0,5],[2,1,3]],]

2 models where the first model has 2 parameter (the first is constant 3 and the second is uniform between 0 and 1) and the second model has 1 lognormal parameter with 0 mean and variance 0.5
[[[0,3,0],[1,0,1]] , [[2,0,0.5],]]

kernel:

3D list.
This list has the same structure as the 'priors', but it has different distributions implemented.

Implemented distributions are:
0 --- uniform distribution.
Example: uniform distribution in the range -2 to 2
[0 , -2 , 2]

1 --- gaussian distribution.
Example : gaussian distribution with mean 0 and variance 0.5
[1 , 0 , 0.5]

***** kwargs *****

fit:

2D list of strings.
This list contains the fitting instructions to define how to fit the experimental data to the systems variables. The first dimension represents the models to be investigated, the second dimension represents the amount of data.

Example:
1 model with 7 species, 3 data series
fit = [['species1+species2' , 'species5' , '3*species4-species7'],]
2 models with each 7 species, 2 data series
fit = [['species1' , 'species2*species3'] , ['species1' , 'species2*species4']]

The default value is 'None', i.e. the order of data series corresponds exactly to the order of species.

beta:

Integer.
It represents the number how often a system is simulated with a sampled parameter

```
          set. For deterministic systems beta is 1, for stochastic systems beta should be
          greater than or equal to 1.

dt:

          Float.
          dt is the internal time step.

restart:

          Boolean variable.
          Start the abc simulator from a given population.
          The default value is 'False'.

source:

        List of strings.
        This length of the list is equal to the number of models to be investigated.
        The entries describe the SBML sources for each model.
        The default value is 'None'.


diagnostic:

          Boolean variable.
          Create diagnostic plots after each population.
          Diagnostic plots are:
          - model distribution (if more than 1 model to analize)
          - all combinations of scatter plots for parameters to be inferred
          The default value is 'False'.

pickling:

          Boolean variable.
          Make a backup after each population that allows to start the abc simulator
          from a given population (see also 'restart').
          The dafault value is 'False'.

plotDataSeries:

          Boolean variable.
          Compute and plot after each population the timeseries for the first 10 sampled
          parameters inclusive data points.
          The dafault value is 'True'.
```

```
full:

          Boolean variable.
          Print status while running the abc simulator.
          The default value is 'False'.

constKernels:

          Boolean variable.
          Do not correct the pertubation kernels in respect to the previous
          population.
          The default value is 'False'.
```

7.2.4 abcodesolve

abcodeint(func, InitValues, timepoints, parameters, dt=0.01, atol=None, rtol=None)

```
Call scipy.integrate.odeint to simulate an ODE system
Return values for each species whose trajectory is described by func
at timepoints given by timepoints.

***** args *****

func:

        a function to integrate, generated by parseInfo.py from an SBML model
        (or equivalently formatted).

InitValues:

        a list of floats representing the initial values of the
        species whose trajectories are described by func.

timepoints:

        a list of times at which values for the species are required.

parameters:

        a tuple of parameters to pass to func

***** kwargs *****

dt:

        Internal timestep.
```

```
        scipy.integrate.odeint is forced to calculate values at this interval.
        For a stiff model a small dt is required for successful simulation.

rtol:

        relative error tolerance.
        For a stiff model a small relative error tolerance
        is required for a successful simulation.

atol:

        absolute error tolerance.
        For a stiff model a small absolute error tolerance
        is required for a successful simulation.
```

7.2.5 checkInputArguments

checkInputABC(ModelName,data,timepoints,numOutput, epsilon, InitValues,integrationType,modelWeights, priors,kernel,fit,restart,source)

```
Check that the information in the input file is consistent with each other and with the model,
and that it is in the format required to run the ABC-SMC algorithm.
Return boolean, string (empty if boolean is True)

***** args *****

ModelName:

            A list of strings

data:

            A numpy array

timepoints:

            A list of floats

numOutputs:

            An integer

epsilon:

        A list of floats
```

```
InitValues:

        An array of floats

integrationType:

        A list of strings

modelWeights:

        A list of floats

priors:

        An list of 3-element-long lists

kernel:

        A list of 3-element-long lists

fit:

        A list of strings

restart:

        Boolean

source:

        A list of strings
```

checkInputSimulation(name, timepoints, InitValues, IntegrationType, ConstantParameters, source, dt)

```
Check that the information in the input file is consistent with each other
and with the model,
and that it is in the format required to simulate the model.
Return boolean, string (empty if boolean is True)

***** args *****

name:
```

```
        A list of strings

timepoints:

        A list of floats

InitValues:

        A list of floats

IntegrationType:

        A list of strings

ConstantParameters:

        An array of floats

source:

        A list of strings

dt:

        A float
```

7.2.6 euclidian

euclidianDistance(data1, data2)

```
Returns the euclidian distance between two data sets.
Data sets must have the same dimensions.

***** args *****

data1, data2:

        numpy array objects with the same dimensions.
```

7.2.7 generateTemplate

generateTemplate(source, filename='input_file_template.txt')

```
Generate a model summary file (model_summary.txt)
and a template file (filename) from one or more SBML source files.
```

```
***** args *****

source:

        a tuple of strings.
        Each tuple entry describes a SBML file.

***** kwargs *****

filename:

        a string.
        The name of the template to be generated.
```

7.2.8 getResults

MatrixToTextfile(matrix,filename,model,eps)

```
Write the part of a three dimensional matrix indexed by model and eps
to a text file

***** args *****

matrix:

        A three-dimensional matrix

filename:

        A string, the name of the file to be written

model:

        An integer

eps:

        An integer
```

printModelDistribution(matrix,eps,filename='model_distribution.txt')

```
Write the contents of a two-dimensional matrix indexed by
eps to a text file.

***** args *****
```

```
matrix:

        A two-dimensional matrix

eps:

        An integer

***** kwargs *****

filename:

        String.
        The name of the text file to be written.
```

plotTimeSeries(ModelName,matrix,integrationType,InitValues,timepoints,fit,
populationNumber=1,dt=0.1,amount=1,model=1,filename='TimeSeries',data=None,plotdata=False)

```
Plot simulated trajectories from the model with accepted parameters.

***** args *****

ModelName:

        Name of the Python file representing the model.

matrix:

        Matrix of accepted parameters

integrationType:

        Integration type for simulating the model.
        One of ODE, SDE or Gillespie

InitValues:

        Initial values for the simulation

timepoints:

        Times at which the simulation should be made.

fit:
```

Fitting instructions for the trajectories from the model.

***** kwargs *****

populationNumber:

Index for the matrix, which population to plot

dt:

Internal timestep for the simulation.

amount:

Number of trajectories to plot.

model:

Index for the matrix, which model to plot parameters for.

filename:

Name of the output file to write to.

data:

Data to plot over the trajectories.

plotdata:

Boolean
Whether or not to overlay the data on top of the trajectories.

getAllHistograms(matrix,weights,population=1,PlotName='AllScatterPlots', model=1)

Plot weighted histograms.

***** args *****

matrix:

Matrix of data

weights:

Weights to assign to data

```
*** kwargs *****

populations:

        Integer, to index matrix with required population

PlotName:

        String.
        Name for the saved plot

model:

        Integer, to index matrix with required model.
```

getAllScatterPlots(matrix,weights,populations=(1,),PlotName='AllScatterPlots', model=1)

```
Plot scatter plots and histograms of data given in matrix.
Used to plot posterior parameter distributions.
***** args *****

matrix:

        Matrix of data.

*** kwargs *****

populations:

        Ordered tuple of integers.
        Determines which data will be plotted.
        Used to index the matrix.

PlotName:

        String
        Name for the saved plot

model:

        Integer
        Determines which data will be plotted.
        Used to index the matrix.
```

getScatterPlot(matrix,parameter,populations=(1,),PlotName='ScatterPlot',model=1)

```
Plot a single scatter plot of accepted parameters.

***** args *****

matrix:

        Matrix of accepted parameters.

parameter:

        List of integers of length 2.
        Used to index the matrix,
        to determine which two parameters to plot against each other.

    ***** kwargs *****

populations:

        Tuple of integers.
        Used to index the matrix.
        Accepted parameters from these populations will be plotted.

PlotName:

        String
        Name for the saved plot

model:

        Integer
        Number for the model from which accepted parameters should be plotted.
```

getModelDistribution(matrix,epsilon,rate,PlotName='ModelDistribution')

```
Plot histograms of the posterior distributions of the models

***** args *****

matrix:

        Matrix containing the model distributions
        after each population.

epsilon:

        Epsilon used for each population
```

```
        (To be displayed above the plot)

rate:

        Acceptance rate for each population
        (To be displayed above the plot)

***** kwargs *****

PlotName:

        Prefix for file name for the saved plot.
        If the saved plot runs over multiple pages,
        each page will be saved to a separate file, with this prefix.
```

7.2.9 parseInfo

getAlgorithmInfo(filename)

```
Parse the user-provided input file
Return all information required to run the ABC-SMC algorithm.

***** args *****

filename:

        String, the name of the user input file
```

getinfoForSimulation(filename)

```
Parse the user-provided input file.
Return all information required to simulate the model.

***** args *****

filename:

        String, the name of the user input file
```

7.2.10 relations

mathMLConditionParser(mathMLstring)

```
Replace and and or with and_ and or_ in a MathML string.
Return the string with and and or replaced by and_ and or_
```

***** args *****

mathMLstring:

A mathMLstring

eq(a, b)

Check for equality between a and b.
Return boolean.

**** args ****

a, b:

Python objects on which a test for equality will work

neq(a, b)

Check for inequality between a and b.
Return boolean.

**** args ****

a, b:

Python objects on which a test for inequality will work

gt(a, b)

Check for inequality a > b.
Return boolean.

**** args ****

a, b:

Python objects on which a test for inequality will work

lt(a, b)

Check for inequality a < b.
Return boolean.

**** args ****

a, b:

        Python objects on which a test for inequality will work

geq(a, b)

Check for inequality a >= b.
Return boolean.

**** args ****

a, b:

        Python objects on which a test for inequality will work

leq(a, b)

Check for inequality a <= b.
Return boolean.

**** args ****

a, b:

        Python objects on which a test for inequality will work

*and_*(a, b)

Logical AND a and b
Return boolean

**** args ****

a, b:

        Python objects on which a Python logical test will work.

*or_*(a, b)

Logical OR a or b
Return boolean

```
**** args ****

a, b:

        Python objects on which a Python logical test will work.
```

piecewise(*args)

```
implements MathML piecewise function passed as args
NB piecewise functions have to be passed through
MathMLConditionParser because they may contain
and and or statements
(which need to be made into and_ and or_ statements)
```

7.2.11 sdeint

sdeint(func, InitValues, parameter, timepoints, dt=0.01)

```
Simulate the function func from the initial values initValues
using the parameters given by parameters as a SDE system.
Return values at the times in timepoints.

***** args *****

func:

        Python function that defines the stochastic system.
        This function takes the species concentration, the systems
        parameter, the actual integration time and the internal time step
        as arguments. It returns the derivatives of the species
        concentrations and the noise terms for each species.

InitValues:

        Tuple of floats.
        The initial concentration for each species.

parameter:

        Tuple of floats.
        Values of parameters passed to function for simulation.

timepoints:

        Ordered tuple of floats.
```

```
        Timepoints at which results of the simulation are returned.

***** kwargs *****

dt:

        Float
        Internal time step.
```

Bibliography

Anderson, R. M. & May, R. M. 1991 *Infections diseases of humans: dynamics and control* Oxford Science Publications.

Gillespie, C. S., Wilkinson, D. J., Proctor, C. J., Shanley, D. P., Boys, R. J. & Kirkwood, T. B. L. 2006 Tools for the SBML Community *Bioinformatics* **22**, 628–629.

Gillespie, D. T. 1977 Exact stochastic simulation of coupled chemical reactions *The Journal of Physical Chemistry* **81**, 2340–2361.

Le Novere, N., Bornstein, B., Broicher, A., Courtot, M. *et al.* 2006 Biomodels database: a free, centralized database of curated, published, quantitative kinetic models of biochemical and cellular systems. *Nucleic Acids Research* **34**, Database issue.

Pritchard, J., Seielstad, M. T., Perez-Lezaun, A. & Feldman, M. W. 1999 Population growth of human Y chromosomes: a study of Y chromosome microsatellites. *Molecular Biology and Evolution* **16**, 1791 – 1798.

Sisson, S. A., Fan, Y. & Tanaka, M. M. 2007 Sequential Monte Carlo without likelihoods *Proceedings of the National Academy of Sciences* **104**, 1760–1765.

Toni, T., Welch, D., Strelkowa, N., Ipsen, A. & Stumpf, M. P. H. 2009 Approximate bayesian computation scheme for parameter inference and model selection in dynamical systems *Journal of the Royal Society Interface* **6**, 187 – 202.

Wilkinson, D. J. 2006 *Stochastic Modelling for Systems Biology* Chapman & Hall/CRC.